Tabela Hash: Índice de Palavras

Configuração 40: E1.1 / H4 / C2 / A4

Enthony Araujo de Oliveira Douglas Patrick Cardozo Freitas

Estrutura de Dados II
Professor: Kennedy Lopes

Nossa Configuração: Nº 40

Código	Definição	Implementação no Projeto
E1.1	Endereçamento aberto fixo	Vetor estático alocado uma única vez com m posições.
H4	Função de Dobra	Conversão para 64 bits e soma de blocos decimais de 4 dígitos.
C2	Sondagem Linear	Busca a próxima posição sequencial: $(h + i) \bmod m$.
A4	Índice de Palavras	Chave (palavra normalizada), valor (frequência e linhas).

Implementação Própria em C11

Sem uso de estruturas prontas (como `dict` ou `unordered_map`). Apenas vetores contíguos em memória, ponteiros nativos e biblioteca padrão.

Aplicação A4: Índice de Palavras

1. Normalização das Chaves

- Letras convertidas para minúsculas.
- Remoção de acentos (á, é, ó, ç → a, e, o, c).
- Descarte de pontuações e símbolos.
- Limite máximo de 255 caracteres por chave.

2. Registro Armazenado

- **Chave:** palavra normalizada.
- **Frequência:** total de ocorrências.
- **Linhas:** vetor dinâmico com números de linha distintos onde a palavra apareceu.

Exemplo Prático

Texto: `"Linha 1: Hash, hash! / Linha 2: Colisão na tabela."`
→ Chave `"hash"`: frequência = 2, linhas = [1].
→ Chave `"colisao"`: frequência = 1, linhas = [2].

Função Hash por Dobra (H4)

A chave textual é transformada em um índice válido $[0, m-1]$ em 3 passos:

1. **Texto $\rightarrow$ Inteiro 64 bits (Base 131):**
   Acumula o código de cada byte com multiplicador primo:
   número = número $\times$ 131 + byte (Exemplo: `"hash"` $\rightarrow$ 235481250)
2. **Fatiamento e Soma (Dobra H4):**
   Divide o número em blocos decimais de 4 dígitos (%10000) e soma:
   $235481250 \rightarrow 2 + 3548 + 1250 = \mathbf{4800}$
3. **Mapeamento para a Tabela:**
   índice = soma mod $m \implies$ Para $m = 2$: $4800 \bmod 2 = \mathbf{0}$.

Determinismo Matemático

A mesma palavra sempre resulta no mesmo índice inicial, respeitando $0 \leq$ índice $< m$.

Sondagem Linear (C2)

Fórmula de Sondagem:

$$p_i(k) = (h(k) + i) \bmod m, \quad \text{para } i = 0, 1, 2, \ldots, m-1$$

Demonstração de Colisão Intencional ($m = 2$)

As palavras `"hash"` e `"tabela"` geram o mesmo índice base: $h = 0$.

Chave	Índice Base (h)	Posição	Comportamento
`"hash"`	0	**0**	Posição 0 vazia → Inserida no índice 0.
`"tabela"`	0	**1**	Posição 0 ocupada → Avança $i = 1$ → Inserida no índice 1.

- Contador de colisões da tabela é incrementado para 1.
- O operador módulo (%m) garante percurso circular fechado sobre o vetor.

Remoção em Endereçamento Aberto: O Marcador REMOVIDO

Cada posição da tabela possui um estado: **VAZIO**, **OCUPADO** ou **REMOVIDO**.

O Erro Clássico

Se ao remover `"hash"` marcássemos a posição 0 como **VAZIO**:

A busca por `"tabela"` começaria no índice 0, veria **VAZIO** e **abortaria prematuramente**, alegando chave inexistente!

A Solução com REMOVIDO

Marcamos a posição 0 como **REMOVIDO**:

- A **Busca** não para: continua avançando até achar a chave ou um **VAZIO**.
- A **Inserção** pode reutilizar o espaço marcado como removido.

Critério de parada da busca: Encontrar a chave, encontrar `VAZIO` ou visitar m posições.

Validação: 8 Testes de Correção Obrigatórios

Todos os casos foram validados pelo menu interativo com capacidade mínima $m = 2$:

Caso de Teste	**Entrada**	**Resultado Obtido**
1. Tabela Vazia	Buscar `"hash"`	Chave não encontrada (OK)
2. Inserção e Busca	Inserir e buscar `"hash"`	Encontrada com frequência 1 (OK)
3. Chave Inexistente	Buscar `"ausente"`	Chave não encontrada (OK)
4. Inserção Duplicada	Inserir `"hash"` novamente	Rejeitada com `RES_DUPLICADA` (OK)
5. Colisão Demonstrada	Inserir `"tabela"`	Inserida no índice 1; colisões = 1 (OK)
6. Tabela Cheia (E1.1)	Inserir terceira chave	Rejeitada com `RES_CHEIA` (OK)
7. Remoção em Colisão	Remover `"hash"`	`"tabela"` continua acessível (OK)
8. Atualização	Atualizar freq de `"tabela"`	Frequência alterada; chave preservada (OK)

Análise Experimental de Desempenho

- **Cenários:** 100, 1.000 e 10.000 chaves sintéticas.
- **Capacidade:** $m = 2n$ (fator de carga constante $\alpha = n/m = 0,50$).
- **Metodologia: 5 repetições** por cenário, medindo tempos de CPU (`clock()`).

n	m	α	**Colisões**	**Inserção (ms)**	**Busca Presente (ms)**	**Busca Ausente (ms)**
100	200	0,50	18	0,021	0,012	0,034
1.000	2.000	0,50	304	0,277	0,114	0,234
10.000	20.000	0,50	4.953	2,542	1,174	3,136

Interpretação dos Tempos

Os tempos medem o lote inteiro de n operações. Para 10.000 chaves, cada busca individual leva em média apenas **0,11 microssegundos**.

Discussão dos Resultados

- **Busca Presente vs. Busca Ausente:**
  A busca por chave existente para no momento em que a chave coincide ($O(1)$ médio).
  A busca ausente precisa percorrer o bloco de colisão até encontrar um **VAZIO**.

- **Agrupamento da Sondagem Linear (C2):**
  Sondagem linear pode formar blocos contíguos (*clustering primário*). Com $\alpha = 0,50$, o espaçamento foi suficiente para garantir alta eficiência.

- **Vetor Estático (E1.1):**
  Capacidade fixa escolhida no início. Não há custo de redimensionamento em tempo de execução.

Demonstração no Terminal

Comandos diretos para demonstrar ao professor:

1. Compilar:

```
gcc -std=c11 -Wall -O2 src/tabela.c src/aplicacao.c src/main.c -o hashapp
gcc -std=c11 -Wall -O2 src/tabela.c src/experimentos.c -o experimentos
```

2. Demonstração de colisão e remoção ($m = 2$):

```
./hashapp 2
```

3. Indexação do texto com arquivo ($m = 64$):

```
./hashapp 64 < dados/demo_config40.txt
```

4. Rodar medições experimentais (5 repetições):

```
./experimentos
```

Conclusão

- **Fidelidade Total à Configuração 40:**
  Estrutura estática ($E1.1$), dobra ($H4$), sondagem linear ($C2$) e índice de palavras ($A4$).
- **Integridade Algorítmica:**
  Uso correto do marcador `REMOVIDO` para manter a cadeia de busca intacta.
- **Validação Prática e Experimental:**
  8 testes de correção validados e medições com 5 repetições comprovando alta velocidade sob $\alpha = 0,50$.

Obrigado!

Estamos prontos para perguntas e demonstração prática.